

Competitive Exam Practice 4 — Answer Key

Registers, Instructions, and the Instruction Cycle

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

Q1 256. Expanding-opcode decomposition:

- **Two-address:** opcode occupies $11 - 4 - 4 = 3$ bits $\Rightarrow 2^3 = 8$ codes; 5 used, leaving 3.
- **One-address:** each of the 3 free two-address prefixes extends into the former second-address field: $3 \times 2^4 = 48$ codes; 32 used, leaving $48 - 32 = 16$.
- **Zero-address:** each of the 16 free one-address prefixes extends into the last 4-bit field: $16 \times 2^4 = 256$ codes.

Q2 16383. 32-bit word = opcode + two register fields + immediate. Each register needs $\lceil \log_2 64 \rceil = 6$ bits, so the two fields cost 12 bits. The opcode needs $\lceil \log_2 45 \rceil = 6$ bits (6 bits encode $64 \geq 45$; 5 bits encode only 32). Remaining immediate width: $32 - 6 - 12 = 14$ bits. Largest unsigned = $2^{14} - 1 = 16383$.

Q3 32. 32-bit word = opcode + mode + two register fields + scalar. Addressing-mode field: $\lceil \log_2 8 \rceil = 3$ bits. Each register: $\lceil \log_2 16 \rceil = 4$ bits, so two fields cost 8 bits. Scalar field: 16 bits. Opcode width = $32 - 3 - 8 - 16 = 5$ bits, giving $2^5 = 32$ unique opcodes per addressing mode.

Q4 (B) 3. On a single-bus data path the execution of `add R0, R1` needs one value on the bus at a time:

- (a) Drive `[R1]` onto the bus into one ALU input.
- (b) Drive `[R0]` onto the bus into the other ALU input.
- (c) ALU computes $R0 \leftarrow R0 + R1$; result written back to `R0`.

No memory access is involved (both operands are in registers), so the two-cycle memory-read timing does not affect this instruction — the minimum is 3 clock cycles.

Q5 (C) 4. `call Rn, sub` is a two-word instruction, so its execution must fetch the second word and then transfer:

- (a) $MAR \leftarrow [PC]$ — load the address of the second word (memory read cycle 1).
- (b) $MDR \leftarrow \text{memory data}$ — fetch the second word (memory read cycle 2).
- (c) $Rn \leftarrow PC + 1$ — ALU computes the return address (saved during fetch of the first word, so PC now points at the second word).
- (d) $PC \leftarrow [MDR]$ — jump target loaded into the PC.

Total: 4 clock cycles.

Q6 3. The XOR-swap uses the identity $x \oplus y$ three times:

- $XOR\ R_1, R_2 \quad (R_1 \leftarrow R_1 \oplus R_2)$
- $XOR\ R_2, R_1 \quad (R_2 \leftarrow (R_1 \oplus R_2) \oplus R_2 = R_1)$
- $XOR\ R_1, R_2 \quad (R_1 \leftarrow (R_1 \oplus R_2) \oplus R_1 = R_2)$

After three XORs the contents are exchanged. No extra register or temporary storage is needed, so 3 is minimal.

Q7 -8. The branch is at instruction $i+3$; the offset is relative to the *next* instruction, i.e. $i+4$. The target is $i+2$, which is 2 instructions before the reference point, i.e. -2 instructions $\times$ 4 bytes per instruction = -8 bytes.

Q8 (A). PUSH decrements first so the operand is stored at the *new* (lower) top: $SP \leftarrow [SP] - 1$ (stack grows downward toward lower addresses), then $M[SP] \leftarrow \text{operand}$. Writing the operand first would overwrite the current top before **SP** moves, and (C)/(D) use $+1$, which would grow the stack upward.